
Universidad del Valle de Guatemala

Computación Paralela y Distribuida

Laboratorio 3

Comunicación bloqueante y no bloqueante con MPI

Estudiante:	Daniel Barillas
Carné:	22193
Sección:	40
Docente (enunciado):	Marlon Fuentes
Entrega vigente:	Sábado 3 de octubre de 2026, 23:59

Informe experimental con ocho capturas originales, cuatro gráficas y 51 mediciones verificadas.
Preparado el 2 de octubre de 2026.

Resumen

Este informe documenta el diseño de cuatro programas de comunicación punto a punto con MPI. Los dos primeros emplean llamadas bloqueantes para resolver un intercambio Ping-Pong y un anillo con un único token. Los dos restantes estudian comunicación no bloqueante mediante una recepción anticipada con trabajo aritmético y un productor-consumidor que transfiere un arreglo por chunks. Se define un protocolo reproducible de medición con `MPI_Wtime`, validación de datos y repeticiones por configuración. Se analizan 51 ejecuciones realizadas por el estudiante en WSL: tres repeticiones de cada una de 17 configuraciones. El RTT mediano de Ping-Pong aumenta de 0.495 a 50.171 microsegundos entre los mensajes mínimo y máximo. El anillo con llamadas separadas registra 2.242 ms, frente a 3.290 ms con `Sendrecv`; esta diferencia se interpreta considerando el tráfico adicional de centinelas. La recepción anticipada realiza cálculo antes de confirmar la llegada del mensaje, sin medir una aceleración frente a una referencia bloqueante. El pipeline obtiene su menor tiempo mediano, 2.279 ms, con chunks de 65 536 enteros. Se conservan todas las observaciones y se limita la interpretación al entorno y a los tamaños ensayados.

Contenido

1	Introducción y alcance	3
2	Fundamentos de comunicación MPI	3
2.1	Llamadas bloqueantes	3
2.2	Llamadas no bloqueantes	3
2.3	Medición temporal	4
3	Entorno y método experimental	4
3.1	Presentación de consola y trazabilidad	5
4	Parte 1: comunicación con bloqueo	6
4.1	Ping-Pong	6
4.2	Token Ring	9
5	Parte 2: comunicación sin bloqueo	12
5.1	Recepción anticipada	12
5.2	Productor-consumidor: pipeline por chunks	15
6	Discusión metodológica	18
6.1	Qué mide y qué no mide cada experimento	18
6.2	Reproducibilidad y futuras extensiones	18
7	Conclusiones	19
8	Referencias	20
9	Apéndice: ejecución y archivos experimentales	21
9.1	Archivos incorporados y trazabilidad	21
9.2	Comprobación de los requisitos	21

1 Introducción y alcance

MPI (Message Passing Interface) permite que procesos independientes intercambien datos mediante mensajes. Cada proceso conoce su identificador mediante `MPI_Comm_rank` y el tamaño de su comunicador mediante `MPI_Comm_size`. En los cuatro programas se utiliza `MPI_COMM_WORLD`, que agrupa a los procesos iniciados por `mpirun`. La coordinación correcta depende de emparejar origen, destino, etiqueta y comunicador en cada transferencia.

El enunciado divide el trabajo en dos partes: comunicación con bloqueo y comunicación sin bloqueo. Cada parte contiene dos ejercicios. La tabla de evaluación del PDF muestra cuatro partes de 25 %, aunque solo las primeras dos están nombradas en el texto; por ello este informe presenta cuatro secciones de ejercicios sin asignarles oficialmente un porcentaje.

Ejercicio	Comunicación	Evidencia requerida
Ping-Pong	<code>MPI_Send</code> y <code>MPI_Recv</code>	Tiempo frente a tamaño de mensaje
Token Ring	<code>MPI_Sendrecv</code> y variante separada	Recorrido correcto y explicación de diferencias
Recepción anticipada	<code>MPI_Irecv</code> , <code>MPI_Isend</code> , <code>MPI_Test</code>	Trabajo efectuado y tiempo frente a tamaño
Pipeline por chunks	<code>MPI_Isend</code> , recepción por chunk y STOP	Tiempo total frente a tamaño de chunk

2 Fundamentos de comunicación MPI

2.1 Llamadas bloqueantes

En una recepción bloqueante, la llamada retorna cuando el buffer contiene el mensaje recibido. En un envío bloqueante, el retorno indica que el buffer del emisor puede reutilizarse; no implica por sí solo que el receptor haya terminado de procesar los datos. La pareja de llamadas exige un orden compatible entre procesos. Si todos los participantes esperan una operación que nadie puede completar, se produce un bloqueo circular.

`MPI_Sendrecv` combina en una sola llamada un envío y una recepción. La operación es bloqueante, pero el subsistema MPI coordina ambos lados y facilita desplazamientos en topologías como un anillo. Los buffers de envío y recepción deben ser distintos. La alternativa con `MPI_Send` y `MPI_Recv` separados requiere planear explícitamente el orden.

2.2 Llamadas no bloqueantes

`MPI_Irecv` y `MPI_Isend` crean una petición y retornan antes de que necesariamente termine la transferencia. `MPI_Test` consulta una petición sin esperar hasta su finalización. Esto permite alternar comunicación y cálculo dentro del mismo proceso. Una petición de envío debe completarse antes de modificar o liberar su buffer.

2.3 Medición temporal

`MPI_Wtime` entrega segundos transcurridos según el reloj local del proceso. Cada duración se calcula restando dos lecturas tomadas en el *mismo* rank; no se restan marcas temporales de ranks distintos. Se realiza una ronda de calentamiento fuera del cronómetro en Ping-Pong y se realizan tres repeticiones por configuración. La mediana reduce la influencia de una corrida aislada sobre el resumen, pero no sustituye la presentación de la dispersión.

3 Entorno y método experimental

Las pruebas se realizaron localmente en Ubuntu bajo WSL con Open MPI. El código utiliza C11 y se compila mediante `mpicc`, con optimización `-O2` y advertencias `-Wall -Wextra -Wpedantic`. El entorno disponible se consultó durante la preparación del informe, sin volver a ejecutar los experimentos:

Componente	Entorno consultado
Sistema	Ubuntu 22.04.3 LTS sobre WSL2
Kernel	6.18.33.2-microsoft-standard-WSL2
MPI	Open MPI 4.1.2, tanto <code>mpicc</code> como <code>mpirun</code>
Procesador	Intel Core i5-10210U, frecuencia nominal 1.60 GHz
Topología visible	4 núcleos, 2 hilos por núcleo; 8 CPU lógicas
Generación de gráficas	Python 3.13.1 y Matplotlib 3.10.1 en Windows

Esta consulta describe el entorno instalado, no una traza histórica de carga, frecuencia o afinidad de cada corrida. El comando del anillo incluye `-oversubscribe`; esa opción permite superar los slots que informe MPI, pero no demuestra que efectivamente se hayan excedido ni garantiza un núcleo dedicado por rank.

Para cada experimento se mantienen constantes los parámetros de control definidos en la tabla metodológica. En recepción anticipada se fija el lote, no la cantidad total de cálculo. Los programas imprimen una línea CSV al final, además de un resumen legible. Las capturas documentan la ejecución y los CSV conservan los números exactos para las gráficas. Las 51 filas de los CSV coinciden con la última línea de sus registros originales locales. Hay 17 configuraciones completas, con repeticiones 1, 2 y 3, sin filas faltantes ni duplicadas. Las capturas corresponden a demostraciones adicionales: sus tiempos no se mezclan con el barrido.

Las tablas presentan mediana, media, desviación estándar muestral (DE) y mínimo-máximo. La DE se calcula con divisor $n - 1$, con $n = 3$. No se eliminó ninguna observación. Las figuras muestran las tres corridas, la mediana y el rango observado; las barras *no* son intervalos de confianza. Las líneas entre medianas son guías de lectura, no modelos ajustados. Los valores se redondean a tres decimales para presentación.

El análisis es descriptivo: con tres repeticiones por configuración no se afirma significancia estadística ni una mejora generalizable. El script `analizar_resultados.py` valida los datos y reproduce las tablas; `graficas_resultados.py` produce las cuatro figuras sin lanzar procesos MPI ni modificar los CSV.

3.1 Presentación de consola y trazabilidad

Los cuatro programas comparten una presentación textual en terminal: encabezados azules, rutas de mensajes cian, comprobaciones verdes y notas amarillas. Los diagramas explican el recorrido de los datos, pero no representan una escala física de tiempo. Las barras de veinte celdas indican finalización o visitas verificadas; la medición numérica siempre aparece por separado. La variable `NO_COLOR` elimina los códigos ANSI cuando se preparan registros de texto. La última línea se conserva como CSV puro para evitar transcribir tiempos desde las capturas.

La implementación separa la lógica MPI de la presentación mediante `common.h` y `visual.h`. Cada programa documenta sus argumentos, roles de los ranks, etiquetas, buffers, orden de llamadas, verificación y límites de la región cronometrada. Los dibujos se imprimen *después* de calcular el tiempo con `MPI_Wtime`; así no alteran las comparaciones solicitadas.

Experimento	Variable independiente	Control
Ping-Pong	1, 64, 1024, 16384, 65536 enteros	1000 rondas, 2 ranks
Token Ring	Dos variantes de comunicación	5 ranks, 1000 vueltas
Recepción anticipada	Los mismos tamaños de mensaje	1000 operaciones por consulta, 2 ranks
Pipeline	Chunks de 256 a 65536 enteros	1 048 576 enteros totales, 2 ranks

4 Parte 1: comunicación con bloqueo

4.1 Ping-Pong

```
dani@LAPTOP-8D4D5EC5:/mnt/c/Users/Daniel Barillas/Desktop/Lab-3_Paralela/Laboratorio-3_Daniel-Barillas_22193_CPD_Sec-40$ mpirun -np 2 ./build/ping_pong 10 1
```

LABORATORIO 03 / EJERCICIO 01

```

PING-PONG
Comunicación bloqueante | 2 procesos

  — Ruta de una ronda —————
[RANK 0] — PING, tag 11 —————> [RANK 1]
[RANK 0] <———— PONG, tag 12 — [RANK 1]

  — Configuración y verificación —————
Rondas solicitadas : 10
Enteros por mensaje: 1
Bytes por mensaje  : 4
Rondas completas   [#####]
✓ El último mensaje regresó sin cambios.
  — Medición con MPI_Wtime —————
Tiempo total       : 0.000010901 s
Ida y vuelta media  : 1.090 us
Ida estimada (RTT/2): 0.545 us
Nota: la estimación RTT/2 supone trayectos simétricos.

tipo,elementos,bytes,rondas,total_s,ida_vuelta_us,ida_estimada_us
ping_pong,1,4,10,0.000010901,1.090,0.545
dani@LAPTOP-8D4D5EC5:/mnt/c/Users/Daniel Barillas/Desktop/Lab-3_Paralela/Laboratorio-3_Daniel-Barillas_22193_CPD_Sec-40$

```

Figura 1. Ping-Pong: 10 rondas con un entero. El mensaje regresa sin cambios.

```
dani@LAPTOP-8D4D5EC5:/mnt/c/Users/Daniel Barillas/Desktop/Lab-3_Paralela/Laboratorio-3_Daniel-Barillas_22193_CPD_Sec-40$ mpirun -np 2 ./build/ping_pong 1000 1024
```

LABORATORIO 03 / EJERCICIO 01

```

PING-PONG
Comunicación bloqueante | 2 procesos

  — Ruta de una ronda —————
[RANK 0] — PING, tag 11 —————> [RANK 1]
[RANK 0] <———— PONG, tag 12 — [RANK 1]

  — Configuración y verificación —————
Rondas solicitadas : 1000
Enteros por mensaje: 1024
Bytes por mensaje  : 4096
Rondas completas   [#####]
✓ El último mensaje regresó sin cambios.
  — Medición con MPI_Wtime —————
Tiempo total       : 0.003557255 s
Ida y vuelta media  : 3.557 us
Ida estimada (RTT/2): 1.779 us
Nota: la estimación RTT/2 supone trayectos simétricos.

tipo,elementos,bytes,rondas,total_s,ida_vuelta_us,ida_estimada_us
ping_pong,1024,4096,1000,0.003557255,3.557,1.779
dani@LAPTOP-8D4D5EC5:/mnt/c/Users/Daniel Barillas/Desktop/Lab-3_Paralela/Laboratorio-3_Daniel-Barillas_22193_CPD_Sec-40$

```

Figura 2. Ping-Pong: 1000 rondas con 1024 enteros, equivalentes a 4096 bytes por mensaje.

Diseño, medición y corrección

El rank 0 genera un buffer de enteros y lo envía a rank 1. Este lo recibe y lo devuelve sin modificarlo. Una ronda consta de un envío y una devolución; el usuario fija N mediante el primer argumento. Para cumplir simultáneamente el caso básico y el estudio de tamaños, el segundo argumento controla el número de enteros: con valor 1 se envía un entero, y con valores mayores se envía un arreglo de enteros. El último mensaje devuelto se compara con el patrón original antes de informar que la prueba fue correcta.

La implementación separa las dos perspectivas en `round_as_origin` y `round_as_echo`. La salida dibuja PING y PONG con etiquetas diferentes, muestra N rondas completadas y presenta el tiempo solo después de verificar el mensaje. Esto permite relacionar la captura con las llamadas MPI del código.

La duración total medida en rank 0 se divide entre N para obtener la latencia media de ida y vuelta:

$$T_{\text{RTT}} = \frac{t_{\text{fin}} - t_{\text{inicio}}}{N}, \quad T_{\text{ida,estimada}} = \frac{T_{\text{RTT}}}{2}.$$

La segunda expresión es una estimación bajo simetría aproximada, no una medición independiente de una dirección. La gráfica utiliza bytes por mensaje en el eje horizontal y T_{RTT} en el vertical.

Resultados del barrido

Las tres repeticiones de cada tamaño usan 1000 rondas y dos ranks. Las 1000 rondas aportan un promedio por corrida; no representan 1000 muestras estadísticas independientes.

Enteros	Bytes	Mediana (μs)	Media $\pm$ DE (μs)	Mín.–máx. (μs)
1	4	0.495	0.545 ± 0.118	0.461–0.680
64	256	0.578	0.579 ± 0.013	0.566–0.592
1024	4096	3.625	3.824 ± 0.489	3.467–4.381
16384	65536	13.846	13.717 ± 0.316	13.357–13.947
65536	262144	50.171	49.659 ± 2.756	46.683–52.124

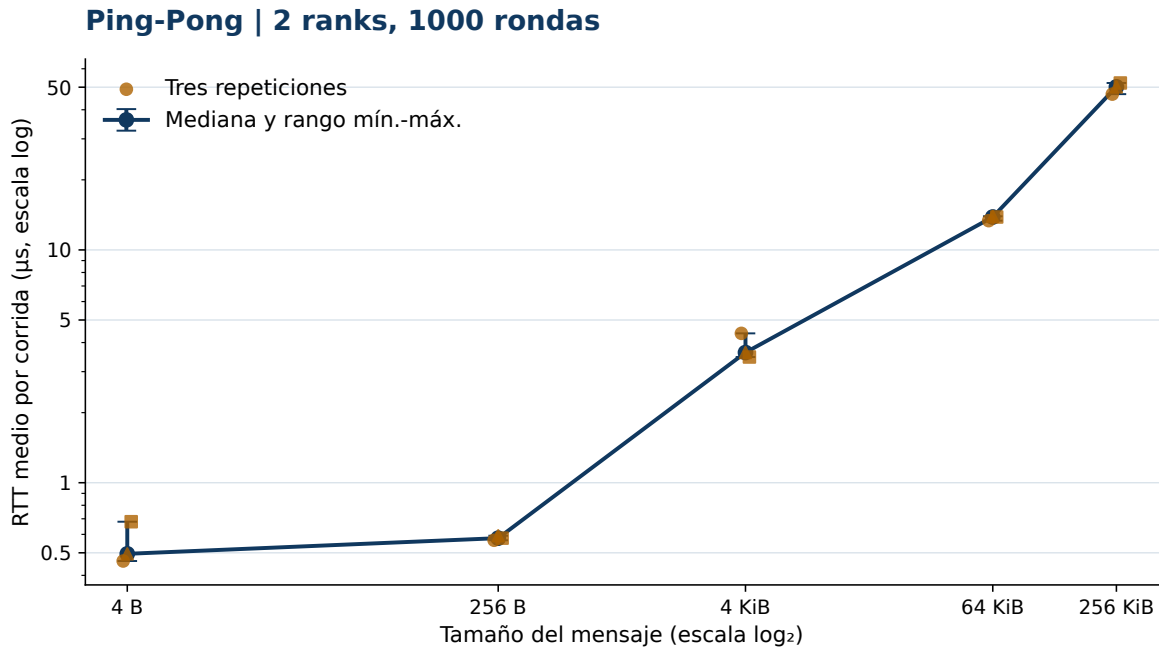


Figura 3. RTT frente a tamaño del mensaje. Ambos ejes son logarítmicos; se muestran las tres corridas y su rango observado.

Interpretación

El RTT mediano aumenta en los cinco tamaños ensayados. Entre 4 y 256 bytes pasa de 0.495 a 0.578 μs ; a 262144 bytes alcanza 50.171 μs . La razón entre extremos es 101.36, aunque el tamaño del mensaje crece 65536 veces. Esto es compatible con un costo fijo por intercambio y otro dependiente del tamaño, pero no se estimó cada componente por separado ni se identificaron umbrales internos de protocolos MPI.

El mensaje de un entero muestra una media superior a su mediana debido a la corrida de 0.680 μs . Se mantiene ese valor en la tabla y en la figura. En el tamaño máximo, el rango 46.683–52.124 μs muestra que el costo depende también de la variación entre ejecuciones. Los valores no son mediciones de una red entre computadoras distintas.

Las capturas iniciales acreditan el caso básico y el mensaje mayor; el análisis cuantitativo utiliza `ping_pong.csv`. La validación comprueba todo el último eco contra $\text{valor}[i] = i \bmod 1009$.

4.2 Token Ring

```

dani@LAPTOP-8D4D5EC5:/mnt/c/Users/Daniel Barillas/Desktop/Lab-3_Paralela/Laboratorio-3_Daniel-Barillas_22193_CP0_Sec-40$ mpirun --oversubscribe -np 5 ./build/token_ring send_recv 1
LABORATORIO 03 / EJERCICIO 02

TOKEN RING
Comunicación bloqueante | más de 4 procesos

— Topología del anillo —
[0] → [1] → [2] → [3] → [4] → [0]
src(i) = (i - 1 + size) % size
dst(i) = (i + 1) % size
— Configuración —
Modo          : send_recv
Procesos      : 5
Vueltas       : 1
Token inicial : 0 en rank 0
— Visitas del token —
Rank 0 [#####]
1 / 1 visitas
Rank 1 [#####]
1 / 1 visitas
Rank 2 [#####]
1 / 1 visitas
Rank 3 [#####]
1 / 1 visitas
Rank 4 [#####]
1 / 1 visitas
✓ Cada rank recibió el token exactamente una vez por vuelta.
— Resultado y tiempo —
Token final      : 5 (esperado: 5)
Tiempo en rank 0 : 0.000460840 s
Send y Recv requieren un orden seguro entre ranks.

tipo,modo,procesos,vueltas,token_final,tiempo_rank0_s
token_ring,send_recv,5,1,5,0.000460840
dani@LAPTOP-8D4D5EC5:/mnt/c/Users/Daniel Barillas/Desktop/Lab-3_Paralela/Laboratorio-3_Daniel-Barillas_22193_CP0_Sec-40$

```

Figura 4. Anillo con envíos y recepciones separados.

```

dani@LAPTOP-8D4D5EC5:/mnt/c/Users/Daniel Barillas/Desktop/Lab-3_Paralela/Laboratorio-3_Daniel-Barillas_22193_CP0_Sec-40$ mpirun --oversubscribe -np 5 ./build/token_ring sendrecv 1
LABORATORIO 03 / EJERCICIO 02

TOKEN RING
Comunicación bloqueante | más de 4 procesos

— Topología del anillo —
[0] → [1] → [2] → [3] → [4] → [0]
src(i) = (i - 1 + size) % size
dst(i) = (i + 1) % size
— Configuración —
Modo          : sendrecv
Procesos      : 5
Vueltas       : 1
Token inicial : 0 en rank 0
— Visitas del token —
Rank 0 [#####]
1 / 1 visitas
Rank 1 [#####]
1 / 1 visitas
Rank 2 [#####]
1 / 1 visitas
Rank 3 [#####]
1 / 1 visitas
Rank 4 [#####]
1 / 1 visitas
✓ Cada rank recibió el token exactamente una vez por vuelta.
— Resultado y tiempo —
Token final      : 5 (esperado: 5)
Tiempo en rank 0 : 0.000204118 s
Sendrecv coordina envío y recepción en una llamada.

tipo,modo,procesos,vueltas,token_final,tiempo_rank0_s
token_ring,sendrecv,5,1,5,0.000204118
dani@LAPTOP-8D4D5EC5:/mnt/c/Users/Daniel Barillas/Desktop/Lab-3_Paralela/Laboratorio-3_Daniel-Barillas_22193_CP0_Sec-40$

```

Figura 5. Anillo con la instrucción MPI_Sendrecv.

Topología y protocolo

Se usan al menos cinco ranks. Para un rank i de un comunicador de tamaño P , el vecino anterior y el siguiente son

$$\text{src}(i) = (i - 1 + P) \bmod P, \quad \text{dst}(i) = (i + 1) \bmod P.$$

Rank 0 comienza con un único token válido. Cada recepción incrementa el valor, lo cual permite verificar que el token visitó todos los procesos. Tras V vueltas, el token devuelto a rank 0 debe valer PV , y cada rank debe registrar exactamente V visitas.

En la primera variante, rank 0 envía para iniciar cada vuelta y después recibe el token de su vecino anterior. Los otros ranks reciben primero y luego envían. Este orden evita que todos queden esperando dentro de un envío bloqueante. En la segunda variante, todos los ranks ejecutan `MPI_Sendrecv` en cada paso: el rank que no posee el token envía un centinela -1 , mientras el poseedor envía el valor válido. Una vuelta consta de P pasos; sigue existiendo un único token válido.

La diferencia central es que `MPI_Sendrecv` combina las dos operaciones bloqueantes en una llamada y coordina su progreso. Con `MPI_Send` y `MPI_Recv` separados, el programador debe establecer el orden seguro de cada rank. El centinela hace explícito el estado vacío del anillo y permite que todos llamen a `MPI_Sendrecv` en cada paso, aunque solo un mensaje contenga el token verdadero.

El resumen visual dibuja la ruta $0 \rightarrow 1 \rightarrow \dots \rightarrow P - 1 \rightarrow 0$ y una barra de visitas por rank. Las barras confirman el contador esperado, mientras el valor numérico final detecta un salto omitido o duplicado.

Resultados y carga de mensajes

El barrido utiliza cinco procesos y 1000 vueltas. Las seis filas registran token final 5000; el programa comprueba además 1000 visitas por rank. El tiempo local de rank 0 incluye la barrera final y excluye el `MPI_Gather` posterior que reúne las visitas.

Modo	Mediana (ms)	Media $\pm$ DE (ms)	Mín.–máx. (ms)
<code>send_recv</code>	2.242	2.245 ± 0.122	2.125–2.369
<code>sendrecv</code>	3.290	3.314 ± 0.211	3.116–3.535

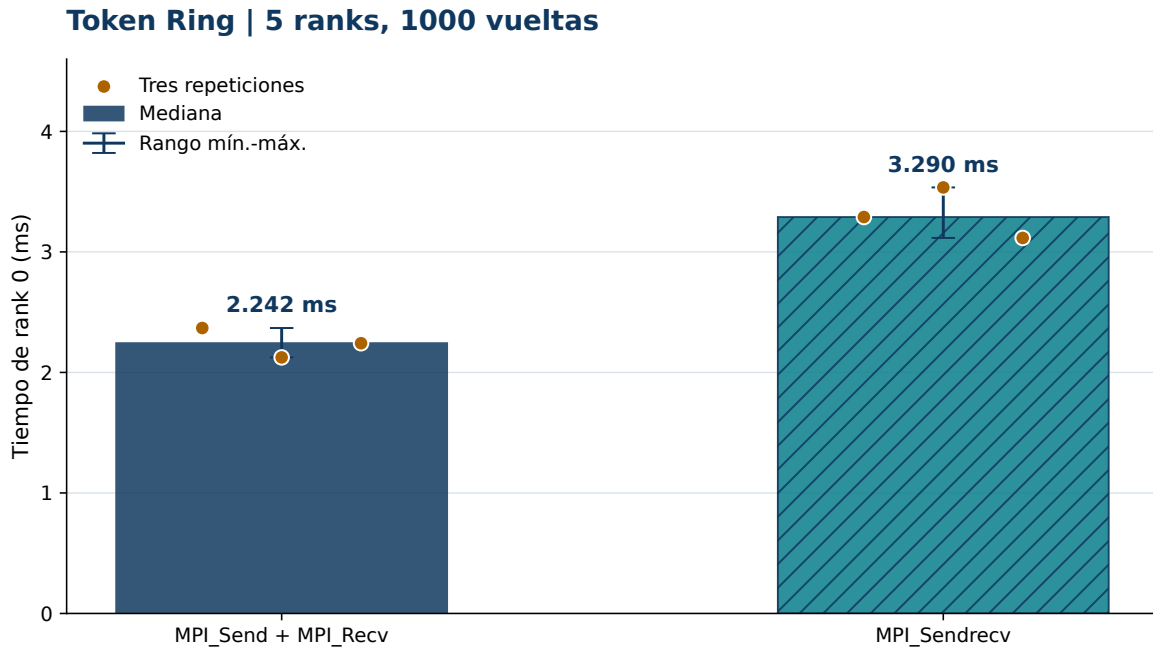


Figura 6. Comparación descriptiva del anillo. Las implementaciones conservan un token válido, pero generan diferente cantidad de mensajes.

Diferencia funcional y diferencia temporal

La mediana de Sendrecv es 1.467 veces la de las llamadas separadas, equivalente a 46.71 % más tiempo. No es una comparación aislada de primitivas equivalentes. Con llamadas separadas se transfieren cinco mensajes por vuelta, es decir, 5000 mensajes de token. En Sendrecv hay cinco pasos por vuelta y cinco envíos por paso: 25 000 mensajes, incluidos los centinelas. Se excluyen de estas cuentas las barreras y la recopilación de visitas.

La variante Sendrecv simplifica la coordinación del desplazamiento, pero su implementación efectúa más trabajo de comunicación. El resultado no demuestra que la primitiva Sendrecv sea intrínsecamente más lenta ni que pierda su utilidad para evitar dependencias cíclicas. Las dos variantes cumplen la misma propiedad lógica del recorrido, no la misma carga de mensajes. Las capturas de una vuelta muestran una visita por rank y token final 5 en ambos modos.

5 Parte 2: comunicación sin bloqueo

5.1 Recepción anticipada

```

dani@LAPTOP-8D4D5EC5:/mnt/c/Users/Daniel_Barillas/Desktop/Lab-3_Paralela/Laboratorio-3_Daniel-Barillas_22193_CPD_Sec-40$ mpirun -np 2 ./build/recepcion_anticipada 1 1000

LABORATORIO 03 / EJERCICIO 03

RECEPCIÓN ANTICIPADA
Comunicación sin bloqueo | 2 procesos

— Línea de tiempo conceptual —————
RANK 0: [preparar] — Isend —————> dato
RANK 1: [Irecv] — [cálculo] — [MPI_Test] — [listo]

— Configuración —————
Enteros recibidos      : 1
Bytes del mensaje      : 4
Operaciones por test   : 1000
— Resultado verificado —————
Recepción              [#####]
✓ Todos los enteros coinciden con el patrón esperado.
Primer entero          : 0
Suma del mensaje       : 0
Trabajo aritmético     : 2000 operaciones
Estado del cálculo     : 1051051757
— Tiempo local de cada rank —————
Recepción + trabajo    : 0.000006601 s (rank 1)
Petición de envío      : 0.000002600 s (rank 0)
Nota: recepción + trabajo no es latencia pura de red.

tipo,elementos,bytes,operaciones_por_test,operaciones_total,tiempo_recepcion_s,tiempo_envio_s,suma
recepcion_anticipada,1,4,1000,2000,0.000006601,0.000002600,0
dani@LAPTOP-8D4D5EC5:/mnt/c/Users/Daniel_Barillas/Desktop/Lab-3_Paralela/Laboratorio-3_Daniel-Barillas_22193_CPD_Sec-40$

```

Figura 7. Recepción básica de un entero y trabajo aritmético.

```

dani@LAPTOP-8D4D5EC5:/mnt/c/Users/Daniel_Barillas/Desktop/Lab-3_Paralela/Laboratorio-3_Daniel-Barillas_22193_CPD_Sec-40$ mpirun -np 2 ./build/recepcion_anticipada 16384 1000

LABORATORIO 03 / EJERCICIO 03

RECEPCIÓN ANTICIPADA
Comunicación sin bloqueo | 2 procesos

— Línea de tiempo conceptual —————
RANK 0: [preparar] — Isend —————> dato
RANK 1: [Irecv] — [cálculo] — [MPI_Test] — [listo]

— Configuración —————
Enteros recibidos      : 16384
Bytes del mensaje      : 65536
Operaciones por test   : 1000
— Resultado verificado —————
Recepción              [#####]
✓ Todos los enteros coinciden con el patrón esperado.
Primer entero          : 0
Suma del mensaje       : 8165256
Trabajo aritmético     : 2000 operaciones
Estado del cálculo     : 1051051757
— Tiempo local de cada rank —————
Recepción + trabajo    : 0.000045308 s (rank 1)
Petición de envío      : 0.000046308 s (rank 0)
Nota: recepción + trabajo no es latencia pura de red.

tipo,elementos,bytes,operaciones_por_test,operaciones_total,tiempo_recepcion_s,tiempo_envio_s,suma
recepcion_anticipada,16384,65536,1000,2000,0.000045308,0.000046308,8165256
dani@LAPTOP-8D4D5EC5:/mnt/c/Users/Daniel_Barillas/Desktop/Lab-3_Paralela/Laboratorio-3_Daniel-Barillas_22193_CPD_Sec-40$

```

Figura 8. Ejemplo con un mensaje de mayor tamaño.

Publicación anticipada y trabajo aritmético

Rank 1 inicia `MPI_Irecv` antes de hacer aritmética. A continuación ejecuta un lote fijo de operaciones y llama a `MPI_Test`; repite esta secuencia hasta que la petición se completa. Rank 0 prepara el buffer, inicia `MPI_Isend` y espera antes de liberarlo. El caso de un elemento cumple el intercambio de un entero y los arreglos permiten estudiar mensajes mayores sin cambiar el protocolo.

El receptor registra el primer entero, la suma validada, el número total de operaciones aritméticas y el tiempo transcurrido desde antes de publicar la recepción hasta que `MPI_Test` informa su finalización. También se registra por separado el tiempo local de envío del rank 0. El tiempo del receptor incluye cómputo y sondeo; por ello no debe interpretarse como latencia pura de red. La cantidad de trabajo realizado antes de recibir puede cambiar entre corridas según la planificación de procesos.

La línea de tiempo de la terminal distingue el envío de rank 0 de la secuencia `Irecv` → cálculo → `MPI_Test` de rank 1. `do_arithmetic` mantiene visible el estado del cálculo y `verify_and_sum` recorre todos los enteros después de completar la petición.

Resultados temporales

En todas las corridas se utilizan lotes de 1000 iteraciones aritméticas. El contador de operaciones corresponde a iteraciones del bucle, no a FLOP ni a instrucciones de procesador medidas.

Bytes	Mediana (μs)	Media $\pm$ DE (μs)	Mín.–máx. (μs)	Envío (μs)
4	6.701	6.935 ± 0.680	6.402–7.701	2.801
256	7.001	7.601 ± 1.683	6.301–9.502	3.601
4096	19.804	21.171 ± 2.543	19.604–24.105	18.703
65536	48.710	49.442 ± 2.289	47.609–52.008	47.409
262144	121.223	124.056 ± 19.213	106.417–144.528	118.823

La última columna es la mediana del tiempo local de envío de rank 0; las demás columnas describen recepción, cálculo y sondeos de rank 1.

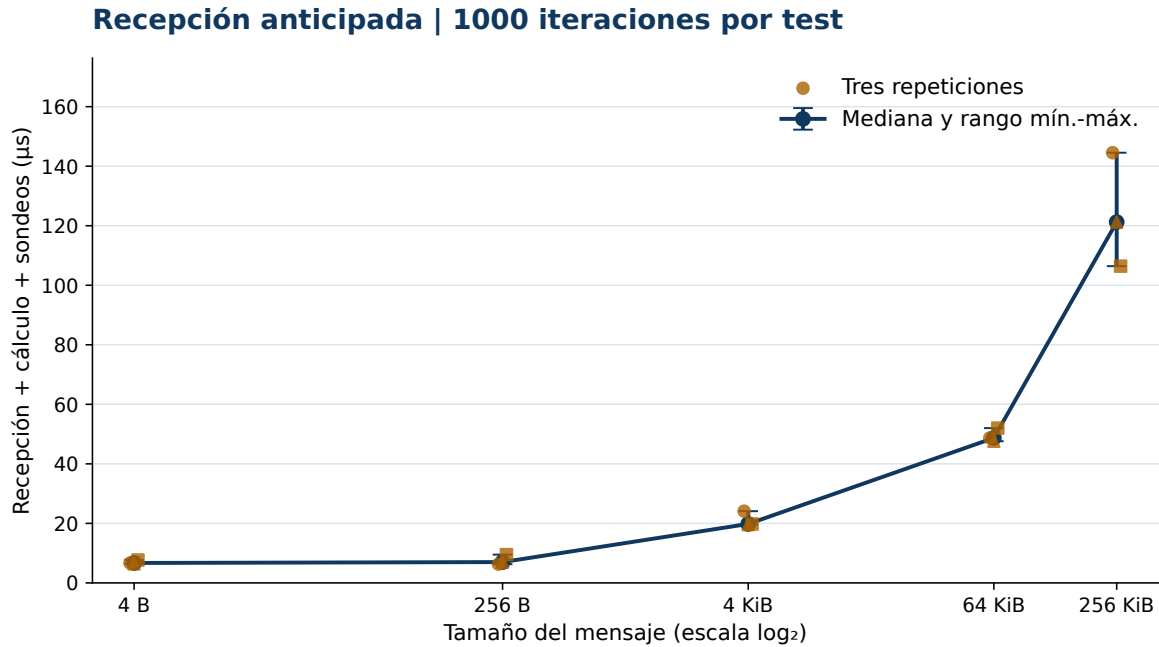


Figura 9. Tiempo del receptor frente a tamaño de mensaje. El eje horizontal es logarítmico y el vertical es lineal.

Trabajo realizado y datos recibidos

Enteros	Iteraciones mín.–máx.	Suma verificada
1	1000–2000	0
64	1000–1000	2016
1024	2000–3000	508641
16384	2000–3000	8165256
65536	1000–5000	33006624

La mediana crece de 6.701 a 121.223 μs , un factor de 18.09. El trabajo total no es constante ni crece monótonamente con el mensaje: a 65 536 enteros se realizan entre 1000 y 5000 iteraciones. Esa variación depende de cuándo el sondeo confirma la petición y no mide por sí sola el rendimiento del transporte. El bucle ejecuta al menos un lote antes de la primera consulta, incluso si el mensaje pudiera completarse antes.

Los tiempos de envío y recepción tienen intervalos locales distintos; restarlos no aísla el costo del cálculo. La evidencia demuestra trabajo entre la publicación y la confirmación de la recepción. Sin una variante bloqueante equivalente y con el mismo trabajo total, no se puede cuantificar una aceleración por solapamiento. Las 15 sumas coinciden con el patrón $i \bmod 1009$, y el estado aritmético visible impide que el resultado del cálculo sea descartado por el compilador.

5.2 Productor-consumidor: pipeline por chunks

```
dani@LAPTOP-8D4D5EC5:/mnt/c/Users/Daniel Barillas/Desktop/Lab-3_Paralela/Laboratorio-3_Daniel-Barillas_22193_CP0_Sec-40$ mpirun -np 2 ./build/pipeline_chunks 1048576 4096
```

LABORATORIO 03 / EJERCICIO 04

```
PIPELINE POR CHUNKS
Productor-consumidor | 2 procesos | STOP con MPI_Isend

— Flujo de datos —
RANK 0 [ARREGLO] → [C0][C1][C2][C3][C4][C5] ... [C255] → RANK 1
RANK 0 [STOP, tag 42] → [fin del bucle]
(Figura ilustrativa; cada C representa un chunk.)

— Configuración —
Enteros totales      : 1048576
Enteros por chunk    : 4096
Bytes por chunk      : 16384
Número de chunks     : 256
Ventana de envíos    : 2 peticiones MPI_Isend

— Verificación del consumidor —
Chunks procesados [#####]
✓ Todos los chunks llegaron en orden y fueron procesados.
✓ El mensaje STOP cerró el bucle de rank 1.
Suma comprobada     : 523641600

— Tiempo total con MPI_Wtime —
Comunicación + trabajo: 0.002891575 s

tipo,elementos_totales,elementos_chunk,bytes_chunk,numero_chunks,tiempo_total_s,suma
pipeline_chunks,1048576,4096,16384,256,0.002891575,523641600
dani@LAPTOP-8D4D5EC5:/mnt/c/Users/Daniel Barillas/Desktop/Lab-3_Paralela/Laboratorio-3_Daniel-Barillas_22193_CP0_Sec-40$
```

Figura 10. Ejecución con chunks, mensaje STOP y suma correcta.

```
dani@LAPTOP-8D4D5EC5:/mnt/c/Users/Daniel Barillas/Desktop/Lab-3_Paralela/Laboratorio-3_Daniel-Barillas_22193_CP0_Sec-40$ mpirun -np 2 ./build/pipeline_chunks 1048576 65536
```

LABORATORIO 03 / EJERCICIO 04

```
PIPELINE POR CHUNKS
Productor-consumidor | 2 procesos | STOP con MPI_Isend

— Flujo de datos —
RANK 0 [ARREGLO] → [C0][C1][C2][C3][C4][C5] ... [C15] → RANK 1
RANK 0 [STOP, tag 42] → [fin del bucle]
(Figura ilustrativa; cada C representa un chunk.)

— Configuración —
Enteros totales      : 1048576
Enteros por chunk    : 65536
Bytes por chunk      : 262144
Número de chunks     : 16
Ventana de envíos    : 2 peticiones MPI_Isend

— Verificación del consumidor —
Chunks procesados [#####]
✓ Todos los chunks llegaron en orden y fueron procesados.
✓ El mensaje STOP cerró el bucle de rank 1.
Suma comprobada     : 523641600

— Tiempo total con MPI_Wtime —
Comunicación + trabajo: 0.002399412 s

tipo,elementos_totales,elementos_chunk,bytes_chunk,numero_chunks,tiempo_total_s,suma
pipeline_chunks,1048576,65536,262144,16,0.002399412,523641600
dani@LAPTOP-8D4D5EC5:/mnt/c/Users/Daniel Barillas/Desktop/Lab-3_Paralela/Laboratorio-3_Daniel-Barillas_22193_CP0_Sec-40$
```

Figura 11. Ejecución con un tamaño de chunk distinto.

Producción, consumo y finalización

Rank 0 crea un arreglo de 1 048 576 enteros. El tamaño de chunk debe dividir al total exactamente; por ello todos los bloques tienen la misma cantidad de elementos. El productor conserva hasta dos envíos `MPI_Isend` pendientes. Antes de reutilizar una petición, espera su finalización. Rank 1 detecta el siguiente mensaje, recibe el chunk y procesa sus enteros de inmediato. Al terminar todos los envíos, rank 0 envía mediante `MPI_Isend` un mensaje de control `STOP`. El receptor lo reconoce por su etiqueta y sale del bucle.

Cada entero se compara con el patrón esperado y se acumula una suma. Al final, el consumidor comunica al productor el número de chunks y su suma. Rank 0 compara ambos contra los valores previstos. El tiempo total se toma en rank 0 y termina después de la confirmación del consumidor y una barrera final; incluye transferencia y procesamiento del lote completo.

La figura textual muestra varios chunks en el trayecto productor-consumidor y un segundo mensaje de control `STOP`. La función `produce` administra las dos peticiones pendientes; `consume` inspecciona etiqueta y tamaño de cada mensaje, procesa el bloque y sale del bucle únicamente tras recibir `STOP`. La figura se genera después de la medición.

El número de chunks es $C = M/B$, donde M es la cantidad total de enteros y B el tamaño de chunk en enteros. Chunks pequeños implican más mensajes y posiblemente mayor costo de coordinación; chunks grandes reducen el número de mensajes, pero pueden limitar el solapamiento. La tendencia concreta debe decidirse con los datos observados.

Resultados del barrido

El volumen total permanece en 1 048 576 enteros de cuatro bytes: 4 194 304 bytes, o 4 MiB. Los tamaños son divisores exactos de ese total y la ventana conserva como máximo dos peticiones de envío.

Enteros/chunk	Chunks	Mediana (ms)	Media $\pm$ DE (ms)	Mín.–máx. (ms)
256	4096	4.691	4.413 ± 0.848	3.462–5.088
1024	1024	3.670	3.667 ± 0.097	3.569–3.762
4096	256	2.844	2.940 ± 0.333	2.666–3.311
16384	64	2.769	3.589 ± 1.824	2.318–5.679
65536	16	2.279	2.271 ± 0.118	2.149–2.385

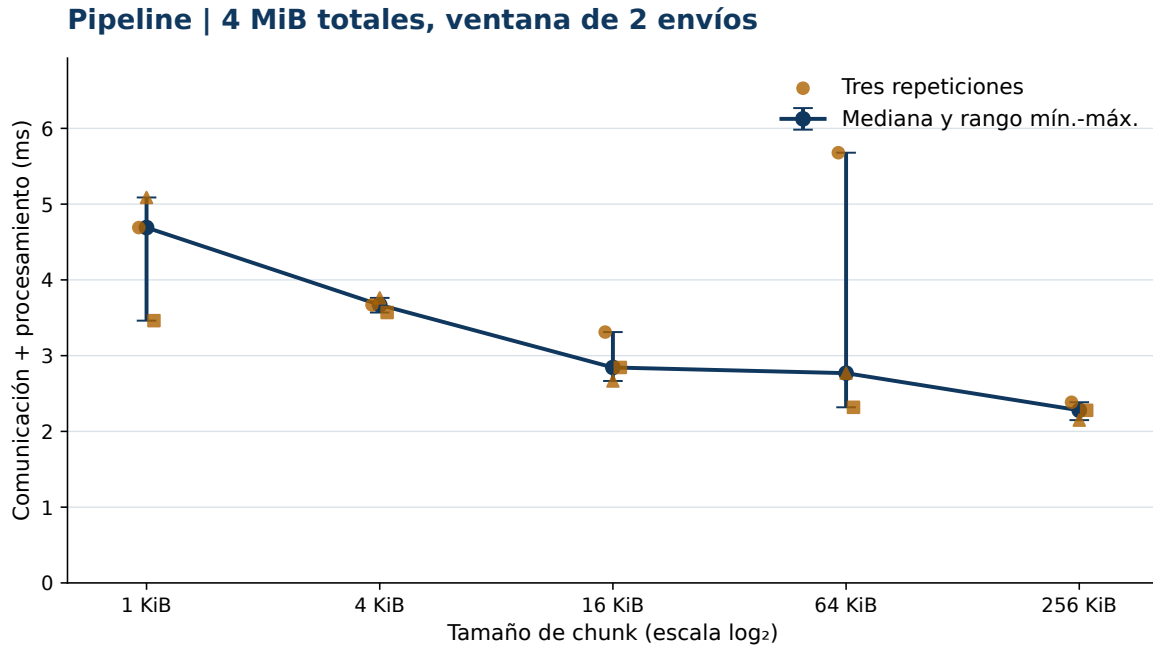


Figura 12. Tiempo total frente a bytes por chunk. Se mantiene un volumen de 4 MiB; las barras muestran el rango de las tres corridas.

Tamaño de bloque, dispersión y corrección

El menor tiempo mediano corresponde a chunks de 65 536 enteros (262 144 bytes), con 2.279 ms. Respecto a los chunks de 256 enteros, el tiempo mediano se reduce 51.42 % y la relación entre duraciones es 2.059. Es una comparación entre tamaños dentro del pipeline, no una aceleración frente a ejecución secuencial. Se identifica el mejor tamaño ensayado, no un óptimo universal.

Los mensajes de datos disminuyen de 4096 a 16 sin alterar el volumen. El comportamiento es compatible con una reducción de la sobrecarga por mensaje; no se cronometraron por separado la transferencia, el procesamiento y la planificación para atribuir causalidad a una sola etapa.

El caso de 16 384 enteros presenta el mayor rango, 2.318–5.679 ms. Su media de 3.589 ms supera a la mediana de 2.769 ms. Se conserva la corrida de 5.679 ms sin diagnosticar una causa no medida. Las medianas decrecen con el tamaño, mientras las medias no son monótonas; por eso se muestran las observaciones individuales además del resumen.

Las 15 sumas son 523 641 600, exactamente la suma de $i \bmod 1000$ para el arreglo completo. El consumidor compara cada elemento, suma en orden y confirma la cantidad de bloques. La señal STOP se completa antes de liberar buffers; las capturas muestran la terminación del bucle y las verificaciones correctas.

6 Discusión metodológica

Los experimentos sobre una sola máquina y WSL pueden reflejar costos de memoria compartida, planificación del sistema operativo y configuración de Open MPI, además de los costos del protocolo. El uso de `-oversubscribe` en Token Ring también puede introducir espera por CPU si se exceden los recursos; la opción por sí sola no demuestra esa condición. Los tiempos entre ejercicios no se comparan como si midieran un mismo recurso. Cada gráfica responde a la pregunta concreta del ejercicio y señala su escala y unidad.

Las salidas verificadas son una condición previa para interpretar los tiempos. En el conjunto recibido no hay filas fallidas ni incompletas y no se descartó ningún resultado. Tres repeticiones permiten describir un mínimo, un máximo y una tendencia central, pero proporcionan poca evidencia sobre estabilidad a largo plazo. Los tamaños se recorren en orden fijo, no aleatorizado; cambios de carga o temperatura durante el barrido podrían influir. No existe una traza de esas condiciones.

6.1 Qué mide y qué no mide cada experimento

Ping-Pong mide un ciclo completo y promedia 1000 rondas. El anillo mide el protocolo de circulación hasta la barrera final. La recepción anticipada mezcla espera, cálculo y consultas locales. El pipeline incluye además procesamiento, comprobación de cada elemento, STOP y confirmación. No deben interpretarse sus diferencias como una clasificación global de primitivas MPI.

El contador de iteraciones no es una tasa de FLOP. La presencia de `Isend` o `Irecv` no demuestra por sí misma transferencia asíncrona autónoma ni una aceleración: describe la interfaz no bloqueante y la posibilidad de continuar trabajo antes de completar la petición. La mejora entre tamaños del pipeline no es *speedup* frente a un programa secuencial. Estas distinciones evitan conclusiones que los datos no respaldan.

6.2 Reproducibilidad y futuras extensiones

Los cuatro CSV se versionan junto con código, capturas y scripts. La reproducción del análisis no requiere nuevas ejecuciones MPI. Las figuras se exportan en PDF vectorial y PNG; el informe incorpora la versión vectorial. Los registros completos del barrido se conservan localmente fuera de Git y se comprobaron contra los CSV.

Una ampliación del experimento podría aumentar repeticiones, aleatorizar el orden, controlar afinidad y carga, y añadir una referencia bloqueante con trabajo idéntico para estudiar el solapamiento. También convendría ensayar más tamaños de chunk y máquinas conectadas por red. Estas posibilidades no se presentan como pruebas ya realizadas.

7 Conclusiones

El diseño cubre los cuatro ejercicios del enunciado con parámetros controlables, comprobaciones de corrección y un protocolo explícito de medición. La comunicación bloqueante exige coordinar el orden de envíos y recepciones; `MPI_Sendrecv` simplifica el desplazamiento circular. Las operaciones no bloqueantes permiten iniciar comunicación y continuar el cálculo o la producción de datos, siempre que se controle la vida de los buffers y la finalización de las peticiones.

1. Ping-Pong conserva el último mensaje y exhibe crecimiento del RTT mediano de 0.495 a 50.171 μ s. El costo no es proporcional únicamente al número de bytes y la mitad del RTT sigue siendo una estimación.
2. Ambas variantes del anillo devuelven token 5000 tras 1000 vueltas con cinco ranks. La variante separada registra 2.242 ms frente a 3.290 ms con `Sendrecv`, pero los 5000 y 25 000 mensajes respectivos impiden atribuir toda la diferencia al costo de la primitiva.
3. La recepción anticipada verifica todas las sumas y realiza trabajo antes de confirmar la recepción. Sus medianas van de 6.701 a 121.223 μ s; estos tiempos incluyen cálculo y sondeos y no cuantifican una aceleración frente a una referencia bloqueante.
4. El pipeline procesa 4 MiB y conserva la suma 523 641 600 en las 15 corridas. El mejor tamaño ensayado es 65 536 enteros por chunk, con 2.279 ms y una reducción mediana de 51.42 % frente al bloque mínimo.
5. Las 51 observaciones se mantienen. La mayor dispersión del pipeline en chunks de 16 384 enteros hace necesario mostrar rango, media y mediana, en lugar de seleccionar solo la corrida más favorable.
6. La corrección del protocolo, la finalización de peticiones y la vida de los buffers son condiciones previas para interpretar rendimiento. Las conclusiones se limitan al equipo, configuraciones y muestra disponibles.

8 Referencias

1. Universidad del Valle de Guatemala. *Laboratorio #3*, Computación Paralela y Distribuida, semestre 2 de 2025. Enunciado PDF proporcionado para esta realización.
2. Open MPI. *MPI_Sendrecv*. Manual oficial.
https://docs.open-mpi.org/en/main/man-openmpi/man3/MPI_Sendrecv.3.html.
3. Open MPI. *MPI_Test*. Manual oficial.
https://docs.open-mpi.org/en/main/man-openmpi/man3/MPI_Test.3.html.
4. Open MPI. *MPI_Wtime*. Manual oficial.
https://docs.open-mpi.org/en/main/man-openmpi/man3/MPI_Wtime.3.html.
5. Kassis, T., Agarwal, V., He, Y., Patel, D. y Brueckner, A. M. (2026). *Scientific Agent Skills: A Library of Procedural Knowledge for Research Agents*. Apoyo metodológico mediante las skills `statistical-analysis` y `matplotlib`.
<https://doi.org/10.48550/arXiv.2609.00065>.

9 Apéndice: ejecución y archivos experimentales

```
make
mpirun -np 2 ./build/ping_pong 10 1
mpirun -oversubscribe -np 5 ./build/token_ring sendrecv 1
mpirun -np 2 ./build/recepcion_anticipada 1 1000
mpirun -np 2 ./build/pipeline_chunks 1048576 4096
bash scripts/mediciones.sh
python3 scripts/analizar_resultados.py
python3 scripts/graficas_resultados.py
```

El barrido de mediciones genera cuatro CSV y conserva las salidas completas de cada corrida. Se detiene si ya existen los CSV para evitar reemplazarlos; un nuevo barrido requiere archivar previamente tanto tablas como registros. Los dos scripts Python analizan los archivos existentes sin ejecutar MPI. El de gráficas requiere Matplotlib; el analizador usa únicamente la biblioteca estándar de Python.

9.1 Archivos incorporados y trazabilidad

Archivo	Uso en este informe
ping_pong.csv	Gráfica de bytes frente a ida y vuelta media
token_ring.csv	Comparación descriptiva de las dos variantes
recepcion_anticipada.csv	Tiempo y trabajo efectuado según bytes
pipeline_chunks.csv	Gráfica de bytes por chunk frente a tiempo total
Capturas PNG	Evidencia visible antes del análisis de cada apartado

Cada CSV incluye tres repeticiones por configuración. Los registros originales de terminal se conservan por separado, de modo que cualquier valor de una figura pueda rastrearse hasta el comando y la ejecución que lo produjo. Las ocho capturas recibidas se conservaron sin alterar su contenido y se nombraron según las referencias de la fuente LaTeX. Las cuatro gráficas proceden exclusivamente del barrido CSV, no de transcribir las demostraciones de consola.

9.2 Comprobación de los requisitos

Requisito	Implementación y evidencia
Dos ranks, entero y N rondas	Ping-Pong parametrizado; capturas, tabla y gráfica de tamaños.
Anillo con más de cuatro ranks	Cinco ranks, vecinos modulares, dos variantes y verificación de visitas.
Recepción y aritmética con Test	Irecv publicado antes del cálculo; contador y estado visibles; gráfica de tamaños.
Chunks iguales y STOP con Isend	Divisibilidad validada, ventana de dos envíos, procesamiento inmediato y suma correcta.
Informe y código fuente	Fuente LaTeX, PDF, cuatro programas documentados, README, CSV y scripts; ejecutables excluidos.